

Lab Exercise 1

Implement a simple MPI program to find $\text{pow}(x, \text{rank})$ for all the processes where x is the integer constant and rank is the rank of the process.

```
5CSED1@dsl-11:~/Documents/240905318_Anivin_PCAP/week4$ mpicc pgm1.c -o pgm1 -lm
5CSED1@dsl-11:~/Documents/240905318_Anivin_PCAP/week4$ mpirun -np 5 ./pgm1
Process 1: pow(2, 1) = 2
Process 2: pow(2, 2) = 4
Process 3: pow(2, 3) = 8
Process 4: pow(2, 4) = 16
Process 0: pow(2, 0) = 1
```

Lab Exercise 2

Implement a program in MPI to toggle the character of a given string indexed by the rank of the process.

Hint: Suppose the string is HELLO and there are 5 processes, then process 0 toggles H to h, process 1 toggles E to e, and so on.

```
5CSED1@dsl-11:~/Documents/240905318_Anivin_PCAP/week4$ mpicc pgm2.c -o pgm2 -lm
5CSED1@dsl-11:~/Documents/240905318_Anivin_PCAP/week4$ mpirun -np 5 ./pgm2
Process 0: H toggled -> h
Process 1: E toggled -> e
Process 2: L toggled -> l
Process 3: L toggled -> l
Process 4: O toggled -> o
```

Lab Exercise 3

Implement a program in MPI where even process prints factorial of the rank and odd ranked process prints ranks Fibonacci number.

```
5CSED1@dsl-11:~/Documents/240905318_Anivin_PCAP/week4$ mpicc pgm3.c -o pgm3 -lm
5CSED1@dsl-11:~/Documents/240905318_Anivin_PCAP/week4$ mpirun -np 8 ./pgm3
Fibonacci value of Process 3: 2
Fibonacci value of Process 1: 1
Factorial of Process 2: 2
Factorial of Process 4: 24
Fibonacci value of Process 5: 5
Fibonacci value of Process 7: 13
Factorial of Process 0: 1
Factorial of Process 6: 720
```

Lab Exercise 4

Implement an MPI program using synchronous send. The sender process sends a word to the receiver. The second process receives the word, toggles each letter of the word and sends it back to the first process. Both processes use synchronous send operations.

```
5CSED1@dsl-11: ~/Documents/240905318_Anivin_PCAP/week4$ mpicc pgm4.c -o pgm4 -lm
5CSED1@dsl-11: ~/Documents/240905318_Anivin_PCAP/week4$ mpirun -np 5 ./pgm4
Process 0 sent: HELLO
Process 1 received: HELLO
Process 0 received: hello
```

Lab Exercise 5

Implement an MPI program where the master process (process 0) sends a number to each of the slaves and the slave processes receive the number and print it. Use standard send.

```
5CSED1@dsl-11: ~/Documents/240905318_Anivin_PCAP/week4$ mpirun -np 5 ./pgm5
Enter a Number: 10
Process 1 received: 10
Process 2 received: 10
Process 3 received: 10
Process 4 received: 10
```

Lab Exercise 6

Implement an MPI program to read an integer value in the root process. Root process sends this value to Process1, Process1 sends this value to Process2 and so on. Last process sends the value back to root process. When sending the value each process will first increment the received value by one. Implement the program using point to point communication routines.

```
5CSED1@dsl-11: ~/Documents/240905318_Anivin_PCAP/week4$ mpicc pgm6.c -o pgm6 -lm
5CSED1@dsl-11: ~/Documents/240905318_Anivin_PCAP/week4$ mpirun -np 5 ./pgm6
Enter an integer: 15
Final value at root: 19
```

Lab Exercise 7

Implement an MPI program to read N elements of the array in the root process (process 0) where N is equal to the total number of processes. The root process sends one value to each of the slaves. Let even ranked process finds square of the received element and odd ranked process finds cube of received element. Use Buffered send.

```
5CSED1@dsl-11:~/Documents/240905318_Anivin_PCAP/week4$ mpicc pgm7.c -o pgm7 -lm
5CSED1@dsl-11:~/Documents/240905318_Anivin_PCAP/week4$ mpirun -np 5 ./pgm7
Enter 5 elements:
5
6
7
8
9
Process 0: 5 -> 25
Process 1: 6 -> 216
Process 2: 7 -> 49
Process 3: 8 -> 512
Process 4: 9 -> 81
```